



1. Généralités

■ DÉFINITION

Une **liste** est une suite d'éléments. Cela peut être une liste d'entiers, par exemple `[5,-7,12,99]`, ou bien une liste de chaînes de caractères, par exemple `["Mars","Avril","Mai"]` ou bien les objets peuvent être de différents types `[3.14,"pi",10e-3,"x",True]`.

2. Manipulations de listes

■ MÉTHODE 1 : CONSTRUCTIONS DE LISTES

Une liste se définit par des éléments entre crochets :

- `liste1 = [5,4,3,2,1]` une liste de 5 entiers,
- `liste2 = ["Vendredi","Samedi","Dimanche"]` une liste de 3 chaînes de caractères,
- `liste3 = []` la liste vide (très utile pour la compléter plus tard!),
- `liste4 = [7]*5` la liste remplie de 7, et de longueur 5.

■ MÉTHODE 2 : ACCÈS À UN ÉLÉMENT

- Pour accéder à un élément de la liste, il suffit d'écrire `liste[i]` où `i` est le rang de l'élément souhaité.
Attention! Le piège c'est que l'on commence à compter à partir du rang 0!
- `liste[-1]` renvoie le dernier élément, `liste[-2]` renvoie l'avant-dernier élément,...

Exemple

Après l'instruction `liste = ["A","B","C","D","E","F"]` alors

- `liste[0]` renvoie "A"
- `liste[1]` renvoie "B"
- `liste[2]` renvoie "C"
- `liste[3]` renvoie "D"
- `liste[4]` renvoie "E"
- `liste[5]` renvoie "F"

	"A"	"B"	"C"	"D"	"E"	"F"
rang :	0	1	2	3	4	5

■ MÉTHODE 3 : AJOUT D'UN ÉLÉMENT

Pour ajouter un élément à la fin de la liste, il suffit d'utiliser la commande `liste.append(element)` (« to append » signifie « ajouter »).

Exemple

Si `premiers = [2,3,5,7]` alors `premiers.append(11)` rajoute 11 à la liste, si ensuite on exécute la commande `premiers.append(13)` alors maintenant la liste `premiers` vaut `[2,3,5,7,11,13]`.

Exemple

Voici comment construire la liste qui contient les premiers carrés :

```
liste_carres = []          # On part d'une liste vide
for i in range(10):
    liste_carres.append(i**2) # On ajoute un carré
```

À la fin `liste_carres` vaut :

`[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`

■ MÉTHODE 5 : CONCATÉNATION DE DEUX LISTES

Si on a deux listes, on peut les fusionner par l'opérateur « + ».

Exemple

Avec `liste1 = [4,5,6]` et `liste2 = [7,8,9]` `liste1 + liste2` vaut `[4,5,6,7,8,9]`.

■ MÉTHODE 6 : AJOUT D'UN ÉLÉMENT À LA FIN

L'opérateur « + » fournit une autre méthode permettant d'ajouter un élément à une liste : `liste = liste + [element]`

Exemple

`[1,2,3,4] + [5]` vaut `[1,2,3,4,5]`.

Attention!

Il faut entourer l'élément à ajouter de crochets.

C'est une méthode alternative à `liste.append(element)`.

■ MÉTHODE 7 : AJOUT D'UN ÉLÉMENT AU DÉBUT

Avec `liste = [element] + liste` on ajoute l'élément en début de liste.

Exemple

`[5] + [1,2,3,4]` vaut `[5,1,2,3,4]`.

■ MÉTHODE 8 : PORTIONS DE LISTES

- `liste[a:]` renvoie la liste des éléments du rang `a` jusqu'à la fin.
- `liste[:b]` renvoie la liste des éléments du début jusqu'au rang `b - 1`.
- On peut extraire d'un seul coup toute une partie de la liste : `liste[a:b]` renvoie la sous-liste des éléments de rang `a` à `b-1`.

"A"	"B"	"C"	"D"	"E"	"F"	"G"
rang : 0	1	2	3	4	5	6

Exemple

- Si `liste = ["A","B","C","D","E","F","G"]` alors
 - `liste[1:4]` renvoie `["B","C","D"]`
 - `liste[0:2]` renvoie `["A","B"]`
 - `liste[4:7]` renvoie `["E","F","G"]`
- On considère la liste ci-dessous :

7	2	4	5	3	10	9	8	3
rang : 0	1	2	3	4	5	6	7	8

Avec `liste = [7,2,4,5,3,10,9,8,3]`, que renvoient les instructions suivantes ?

- `liste[3:5]`
- `liste[:6]`
- `liste[4:]`
- `liste[-1]`

Remarque : D'autres méthodes, moins utilisées mais toutes aussi utiles, seront rencontrées en cours d'année (notamment les méthodes `.pop`, `insert`, `remove`, `.sort`.... Leurs utilisations seront rappelées dans les énoncés, contrairement aux situations précédentes qui doivent être maîtrisées.